

Exercise Sheet 3: k-MST

Mathematical Programming and Optimization in Transport Logistics

Student 1 (student id)

Student 2 (student id)

Student 3 (student id)

SS 2026

-
- Typeset your solution (in, e.g., $\text{\LaTeX}$). You may use the provided `.tex` template for this.
 - Cite all references you used, including any use of generative AI.
 - If you work as a team, list all team members as coauthors in the PDF and the source code (e.g., `pyproject.toml`)
 - Upload your solution in TUWEL before the deadline. It should consist of **two files**
 - a `.pdf` with your solution to the exercises, and
 - an archive (e.g., `.zip`, `.tar.gz`) containing your implementation.
 - Upload **one solution** per team.
 - When formulating an optimization problem as a (mixed) integer linear program, make sure to **define and describe all variables and constraints**. See exercise sheet 1 for examples how this can be done.
 - If you use any additional data structures, make sure to define and describe them as well.
 - Make sure that your implementation matches your formulation in the `.pdf` file.
 - Use directed formulations.
 - Use an artificial root node 0 with root edges $(0, v)$, $\forall v \in V$, in your formulations. (Note that they must be handled specially in your models.)
 - Be aware of your solver's tolerances (e.g., `FeasibilityTol`, `IntFeasTol` in Gurobi) in your separation algorithms.
-

Exercise 3.1 k -MST (10 points)

Definition 1 (k-minimum spanning tree problem, k-MST). Given an undirected graph $G = (V, E)$, non-negative edge weights $w_e \in \mathbb{N}, \forall e \in E$, and a number $k \in \{0, \dots, |V|\}$, find a minimum-weight subtree of G that spans exactly k nodes.

- Design two different exponentially-sized (mixed) integer linear programming formulations for the k-MST problem:
 - one with *cycle elimination constraints* (CEC), and
 - one with *directed cutset constraints* (DCC).
- For each formulation, describe an algorithm that solves the separation problem for the respective family of constraints (CEC or DCC). You may use different algorithms for integral and fractional solutions, but your algorithms *must* be able to solve the separation problem for fractional solutions (rounding your solution values and reusing the separation algorithm for integral solutions does not count).
- Implement your formulations with an ILP solver (e.g., Gurobi). You can use the provided Python template for this, or any other suitable programming language / modelling framework / ILP solver. Note that they must support branch-and-cut.
- Solve the provided ten instances (`g01.dat` – `g10.dat`) for $k = \left\lceil \frac{1}{5}|V| \right\rceil$ and $k = \left\lceil \frac{1}{2}|V| \right\rceil$ with each formulation and report (at least) the following data for each formulation:
 - the number of variables (total, continuous, integer, binary),
 - the number of constraints (at the start),
 - the required runtime and maximum memory consumption,
 - the number of branch-and-cut nodes explored,
 - the number of constraints added by your separation algorithms,
 - the objective value of the best solution you found, and
 - the optimality gap.

For these benchmark runs, use

- a single thread (`--threads 1`),
- a time limit of at least 1 hour (`--timelimit 3600`), and
- a memory limit of at least 8 GB (`--memorylimit 8`)

for all instances.
